

Towards Systematic Maintenance of Assurance for Evolving Self-Adaptive Systems

Logan Murphy¹, Marc Carwehl², Juliane Päßler³, Marsha Chechik¹

Abstract—Despite many advances in self-adaptive systems (SAS) research, the community has only recently begun exploring the *evolution* of SASs. We consider an evolution to be a necessary change of the SAS outside of its self-adaptation capabilities, e.g., to respond to unforeseen changes. In this work, we study the *assurance* of evolving SASs. Dynamic assurance cases (ACs) have been proposed as a means to certify a system’s safety and reliability under uncertainty. However, when the SAS evolves, developers require guidance about which parts of a dynamic AC can be repurposed and which costs may be introduced in re-assuring the evolved SAS. Towards this goal, we propose ReASSURE, a framework which provides a structured approach for reusing assurance evidence for evolving SASs. Building on existing techniques for dynamic AC development, compositional assurance, and product line engineering, ReASSURE aims to maximize reuse, minimize the cost, and quantify the effort of assuring evolving SASs.

Index Terms—Self-Adaptive Systems, Product Lines, Assurance Cases, System Evolution

I. INTRODUCTION

When self-adaptive systems (SASs) are deployed in safety-critical domains, stakeholders require *assurance* that these systems will operate as intended. In industries such as automotive and aerospace, *assurance cases* (ACs) have been adopted as a means to demonstrate system reliability [1]. An AC is a structured model which combines verification evidence, environmental assumptions, and systematic argumentation to provide confidence in a system’s reliability.

To produce ACs for SASs, we identify the following two challenges: on the one hand, design-time uncertainty about the system or its environment prevents the creation of a complete AC [2]; instead, the AC for the system must be developed *dynamically* using information collected at runtime. For example, ENTRUST [2] has been presented as a systematic dynamic assurance framework. On the other hand, unforeseen changes to the SAS or its application domain might require an *evolution* of the SAS. These unforeseen changes may include altered application domains, changes to mission priorities, or improvements in technology, and they may require the introduction or modification of new system features and behaviors. Modifications of SASs may be performed manually (e.g., by adding a new sensor [3]), or remotely (e.g., by delivering a software update over a wireless network [4]). In either case, modifications of a system often impact its assurance; verification artifacts

may be out of date, and new functionalities may introduce new assurance obligations. As SASs and their application domains increase in complexity, the potential for unforeseen changes and, therefore, evolution, increases as well. Given the complexity of producing assurance for SASs, we identify that *systematic methods for re-assuring an SAS after an evolution are lacking*.

To address this gap, this paper proposes ReASSURE, an assurance engineering framework for evolving SASs. More concretely, following an evolution of the SAS, ReASSURE aims to *maximize reuse* of available assurance artifacts, *quantify the effort* for evolving the AC, and *minimize the cost* required for obtaining the necessary evidence. The fundamental principle underlying ReASSURE is a systematic approach to AC design which constrains the assurance impact of system evolutions through compositional contract-based reasoning. ReASSURE also employs the dynamic product line (DPL) [5] formalism which provides a structured approach to modeling variability in an SAS. Product line modeling has also been applied to AC design, providing a formal mapping between the system’s adaptation space and assurance obligations [6]. Leveraging variability modeling introduced by DPLs, we assume in ReASSURE that the variability in the investigated SAS is modeled explicitly as a DPL.

Contributions: This paper makes the following contributions: (1) we propose ReASSURE, an AC development framework for SASs which seeks to maximize reuse, quantify the effort, and minimize the cost of assuring evolving SASs; (2) we provide a roadmap towards implementing ReASSURE, identifying existing tools and techniques that can be integrated into a holistic AC development methodology.

Paper structure: In Sec. II, we provide the necessary background material for ReASSURE. We provide an overview of ReASSURE in Sec. III and discuss ongoing efforts towards implementing it in Sec. IV. We describe related work in Sec. V, before concluding in Sec. VI.

II. BACKGROUND

We begin by recalling the basic principles of dynamic AC development for SASs. We then provide an overview of existing ideas and techniques used in ReASSURE, namely, DPLs and contract-based ACs.

A. Assurance Cases for Self-Adaptive Systems

An *assurance case* (AC) is an auditable document which uses structured argumentation over evidence artifacts to certify a set of goals, e.g., safety, or reliability properties [1].

Reusing Assurance to Support Self-Adaptive System Evolution under Uncertainty with Reduced Effort

¹Logan Murphy and Marsha Chechik are with the University of Toronto, Canada {lmurphy, chechik}@cs.toronto.edu,

²Marc Carwehl is with Universität zu Berlin, Germany m.carwehl@hu-berlin.de

³Juliane Päßler is with the University of Oslo, Norway julipas@ifi.uio.no

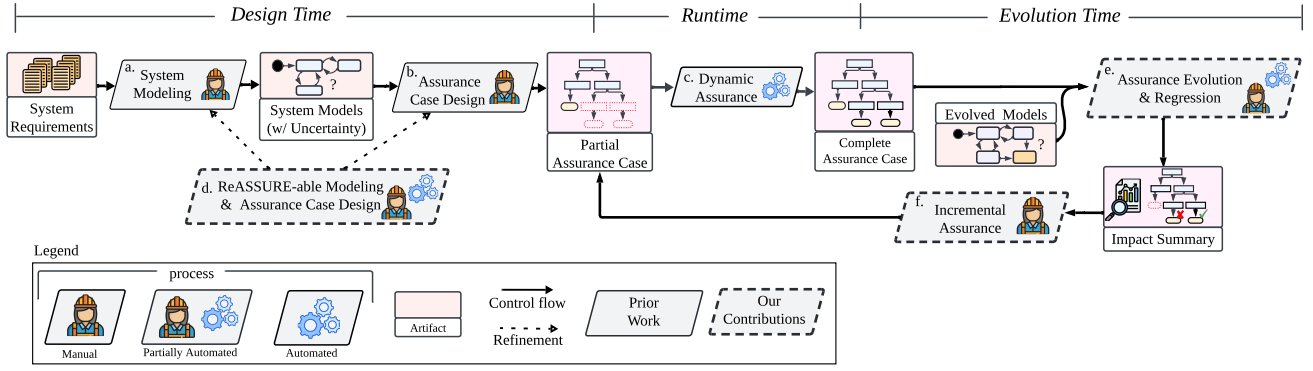


Fig. 1. Extension and refinement of dynamic AC development with ReASSURE.

We focus on ACs developed using languages such as Goal Structuring Notation [7] or the Claims-Argument-Evidence language [8]. In such languages, ACs are represented as trees whose inner nodes are either assurance goals (i.e., claims about the system to be supported) or arguments, and whose leaves contain references to evidence artifacts (e.g., simulation logs, model-checking results).

While SASs can be deployed in the presence of uncertainty, uncertainty also impedes creating a complete AC at design-time. In this work, we consider SASs represented by a two-layered architecture consisting of a *managed system*, which interacts with the system’s environment, e.g., through sensors and actuators, and a *managing system*, which implements the system’s adaptation logic [9]. While a managing system can be modeled, and thus assured, at design-time, a managed system model typically includes uncertainty that might only be resolved at runtime.

Example 1. Consider an autonomous underwater vehicle (AUV) used for maritime surveillance. The AUV uses a sensor to measure the amount of dissolved oxygen at different depths and in different areas of the sea. To assure the system, engineers need to estimate the likelihood of a sensor failure, but an accurate estimation may be infeasible at design time.

Dynamic assurance engineering has been proposed to accommodate uncertainties during design-time, e.g., in the ENTRUST methodology [2]. Fig. 1(a–c) shows an illustration of dynamic AC development, which is separated into design-time and runtime stages. At design-time, engineers define models for the managed system based on its requirements (Fig. 1a). Next, these models are used to create a *partial* AC, which contains all assurance evidence available at design-time and uses predefined *templates* to organize the evidence which will be collected at runtime (Fig. 1b). This evidence can refer to the system and its environment. This new information can then be used to resolve remaining uncertainty in the system models, enabling verification and thereby completing the system’s AC (Fig. 1c). As the system continues to adapt, e.g., to changes in its environment, the AC is continually updated to assure the system.

Example 2. To dynamically assure the AUV described in Example 1, the engineers can use a reconfigurable modeling

formalism, e.g., a parameterized Markov chain, as the basis of a partial AC at design-time. At runtime, the managing system will calculate an accurate failure rate based on real-time observations, and will be able to produce the necessary evidence to complete and maintain the system’s AC.

B. Dynamic Product Lines (DPLs)

DPLs are a formalism for SASs (e.g., [10], [11]), leveraging techniques from product line (PL) engineering. A PL defines a set of systems that differ in some functionalities, modeled as *features*, that can be absent or present in an individual system. Given a set of features F , a *configuration* of a PL is a subset of F . Not all configurations may be valid, e.g., some features may be mutually exclusive, and some features may depend on other features. A *feature model* defines the set of valid configurations of the PL, called *products*. It may be defined diagrammatically or as a propositional expression over F (referred to as a *feature expression*) [12]. Products induce a boolean valuation on feature expressions, where a feature expression is *satisfied* by a product p iff it evaluates to true when all features present in p are mapped to true and all others to false. Feature expressions are commonly used as *presence conditions*, i.e., annotations indicating products in which a given artifact (a line of code, a model element, etc.) exists. A DPL applies these general PL concepts to implement self-adaptation. Following the two-layered architecture for SASs described above, an SAS may be (partially or completely) modeled as a DPL, in which features represent aspects of the managed system or the environment. The managing system is then responsible for (de)activating system features as a means of self-adaptation [13].

Example 3. Fig. 2 visualizes the feature model of the AUV discussed in Example 1. To transmit data, the AUV can surface to communicate with a land-based control center. This communication can be conducted via VHF radio or satellite. However, VHF is only available in coastal waters. Thus, the AUV has to resort to satellite-based communication when it is deployed offshore. Furthermore, the environment is surveyed by analyzing the amount of dissolved oxygen in the water. In the DPL formalism, the managed system

The maximum distance for VHF communication is ≈ 30 nautical miles.

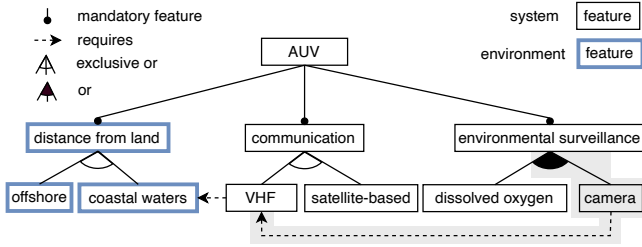


Fig. 2. A feature model for a self-adaptive AUV including an evolution. A new feature introduced in an evolution is shown with gray background.

can be modeled as an annotative PL of transition systems; for instance, a transition corresponding to an oxygen measurement would be guarded by the presence condition `dissolved_oxygen`, indicating that this feature must be active for this transition to be available.

ACs for PLs: In a PL with n features, there can be up to 2^n products, making product-based AC development infeasible. Instead, techniques have been proposed for developing *variational ACs* [6]. In a variational AC, goals are expressed over abstract *products*, and are annotated with a presence condition. A variational AC thus encodes a “PL of ACs”. An AC for product p can be obtained from the variational AC by retaining only the nodes whose presence conditions are satisfied by p . Variational ACs are also given special semantics: in a traditional AC, every assurance goal is either supported or unsupported; in a variational AC, each goal may be supported for some set of products, and unsupported for other products.

C. Assurance Cases from Contract-Based Design

Large, complex systems can benefit from *component-based design with assume-guarantee contracts* [14]. In this work, we consider components to have two interfaces, one containing input ports and one containing output ports. Components can be connected through input and output ports, e.g., an output of one component might provide an input for another component. Environment sensors and system actuators can also be modeled using input and output ports. Components can also be arranged hierarchically, i.e., a complex component C can be *decomposed* into *subcomponents* $C_1, \dots, C_n$. The components $C_1, \dots, C_n$ can be connected through some of their ports. The remaining, non-connected, input ports define C ’s input ports, while the remaining, non-connected output ports define C ’s output ports. We can associate each component with a contract $K = (A, G)$, where A is a set of assumptions and G is a set of guarantees. Both assumptions and guarantees are behavioral specifications over the component’s ports, formalized in an appropriate language (e.g., Metric Temporal Logic). Each assumption is mapped to an input port of the component, and each guarantee is mapped to an output port of the component. A component C *satisfies its contract* if, when deployed in a context where each of its assumptions is satisfied, each of its guarantees is satisfied. Contract satisfaction for *atomic* components can be verified through any number of means, e.g., testing or formal verification. For *complex*

components, contract satisfaction can be shown through *contract refinement*, i.e., by verifying that the contracts of its subcomponents logically refine its own contract, and that each subcomponent satisfies its contract. Contract refinement can be verified using automated tools, e.g., OCRA [15].

Example 4. Suppose a team of engineers is tasked with developing the AUV described in Ex. 3, such that it is intended to operate no more than 20 miles from shore (assumption), and which must send an analysis result to the control center at least once every 15 seconds (guarantee). The engineers formalize this specification as a contract K . To implement the system, they begin by procuring a third-party communications module which has been verified to satisfy a contract K' , covering operations up to 30 miles from shore, and with a throughput of no less than one message every 5 seconds. Since the guarantee of K' is stronger and its assumption is weaker, K' refines K , so the correctness of the communications module implies the correctness of the AUV (with respect to its contract).

ACs for Component-Based Systems: An AC for a system developed via component-based methods can be derived systematically as follows: (1) we begin with some high-level assurance goal, e.g., from stakeholders or through conducting a hazard analysis, which forms the top-level goal of the AC; (2) we define a contract of the system (i.e., the top-level component C) such that providing its guarantees will ensure the top-level goal is satisfied. Thus, the top-level goal is decomposed into a subgoal G_{grt}^C asserting that C provides its guarantees; (3) to support G_{grt}^C , we need to show that the environmental assumptions of C are fulfilled, and that its contract is satisfied. Assuming C is a complex component, the latter can be shown through contract refinement. Thus we decompose G_{grt}^C into: a subgoal G_{env}^C which asserts that C ’s environmental assumptions are satisfied; a subgoal G_{ref}^C which asserts that C ’s contract is refined by its subcomponents; for each of the subcomponents $\{C_1, \dots, C_n\}$ of C , the subgoal $G_{\text{grt}}^{C_i}$. We iterate this process for each C_i with one additional step: for each (non-environmental) assumption A_k of component C_i , we produce a subgoal $G_{A_k}^{C_i}$ indicating which guarantee of which component is supporting A_k . This takes the form of an *away goal*, i.e., a reference to the AC submodule of the supporting component. We eventually reach our atomic components; at this point, we can assure contract satisfaction directly, e.g., via testing or model checking. In this manner, assurance is modularized at the component level, with contract refinement providing compositionality. Thus, we can easily trace which pieces of evidence need to be considered when the system is modified.

Example 5. Consider extending the AUV described in Ex. 1–4 with an additional subcomponent SURV for environmental surveillance, initially responsible only for measuring oxygen levels. Accordingly, the AUV’s guarantees are strengthened to require the production of sufficiently accurate measurements. The SURV component is designed to receive data through an input port to an AUV sensor, compute oxygen

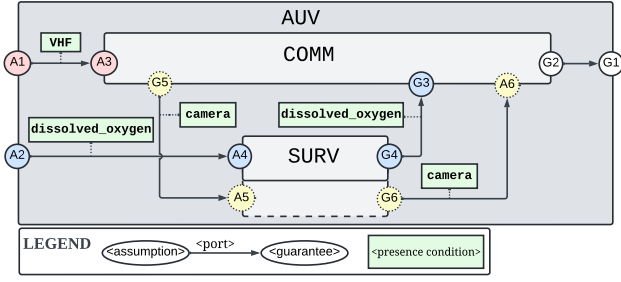


Fig. 3. Component-based DPL architecture for the AUV. Connected ports are represented as edges labelled with presence conditions. Ports labeled with A's indicate assumptions and G's represent guarantees. Elements added as part of an addition of the camera feature are shown with dotted lines.

levels, and relay its analysis results to the COMM component. The SURV component assumes that the data necessary to compute oxygen levels will be available, and guarantees that the measured levels will fall within a certain margin of error, e.g., deviating by at most 1%. To derive a contract-based AC for this component-based system, we create a parent goal G_{Grt}^{AUV} asserting that the AUV component provides its guarantees. This is decomposed into four subgoals: G_{ENV}^{AUV} , asserting that the AUV's environmental assumptions are satisfied; G_{Ref}^{AUV} , asserting that the contracts for COMM and SURV refine that of the AUV; and G_{Grt}^{COMM} and G_{Grt}^{SURV} , which assert that COMM and SURV provide their guarantees, respectively. This in turn is supported by checking that all (non-environmental) assumptions of these components are satisfied, and that they satisfy their own contracts. Since these are atomic components, satisfaction must be checked directly, e.g., via testing or formal verification.

Assuring PLs of Component-Based Systems. We can also model a PL of component-based systems with variability in the components. For instance, a connected pair of input/output ports can be annotated with a presence condition to specify in which products these ports are available. Thus, a product's architecture can be derived from the PL architecture by removing all ports whose presence conditions are not satisfied by the product (components with no ports left are removed entirely). Since ports of components might not be available for a specific product, contract satisfaction has to be checked using only the ports available for the product. It follows that a component may satisfy its contract, or that a refinement may hold, only in some products and not others. In other words, these verification problems become *variational*. Variability-aware methods for producing ACs for (static) component-based PLs have been proposed [16].

Example 6. Recall the component-based architecture described for the AUV in Ex. 1–5. We can augment such an architecture to account for the variability defined in the DPL model. The corresponding variability-aware architecture for the AUV is shown in Fig. 3; ports between components are labelled by presence conditions over features of the AUV.

III. THE ReASSURE FRAMEWORK

While assurance engineering is expensive in any context, these costs are of particular concern for SASs. First, as each adaptation can introduce new verification goals, the work required to maintain assurance of the managed system can grow exponentially [2]. Second, the costs of performing dynamic assurance at runtime can be significant for SASs with resource limitations [17]. It is therefore highly desirable to *reuse* as much assurance evidence as possible through a system evolution, such as the introduction of a new feature, or re-implementation of an existing component. We argue that reuse can be better supported through careful design of the assurance case for the managed system. We propose as our contribution the ReASSURE framework, which is a simultaneous *refinement* and *extension* of dynamic assurance methodologies such as ENTRUST.

Fig. 1 (d–f) illustrates how ReASSURE extends and refines existing dynamic AC methodologies. We refer to ACs developed using ReASSURE as ReASSURE-able ACs. The design stage of ReASSURE *refines* the traditional approach to dynamic AC development with component-based and product line modeling (Fig. 1d). This design stage results in a dynamic AC with specialized structure and semantics well-suited for handling system evolutions. It also defines a method for analyzing the regression of such ACs following system evolutions, such that engineers can glean crucial information about the assurance costs that will be induced (Fig. 1e). Based on this information, engineers can determine which assurance obligations should be re-verified (Fig. 1f), or whether the SAS evolution should be re-designed to reduce costs, prior to redeployment of the evolved system.

A. *Refinement: Designing ReASSURE-able Assurance Cases*

Our framework is used to produce ReASSURE-able ACs, which integrate contract-based and variational AC design principles. More concretely, ReASSURE-able ACs (i) use variational AC modeling to provide an interpretation of assurance goals in terms of the system's adaptation space, and (ii) enable systematic reasoning about the compositionality of assurance through contracts on system components.

Fig. 4 outlines the process of producing a ReASSURE-able AC for an SAS. Compared to traditional dynamic assurance methodologies, system modeling and AC design in ReASSURE are closely intertwined. Beginning with a set of requirements for the SAS, engineer(s) proceed to model the system. In ReASSURE, we distinguish three essential modeling activities: the development of a component-based architecture for the SAS (Fig. 4a), variability modeling for the DPL underlying the SAS (Fig. 4b), and behavioural modeling of system components (Fig. 4c). Behavioral modeling of the SAS is inherited from prior work [2], while component-based and variability modeling are specific to ReASSURE. Variability modeling includes modeling the system's adaptation space in terms of the environment and system features, defining its feature model, and providing a variability encoding (e.g., presence conditions) on implementation artifacts. In particular, variability of the component-

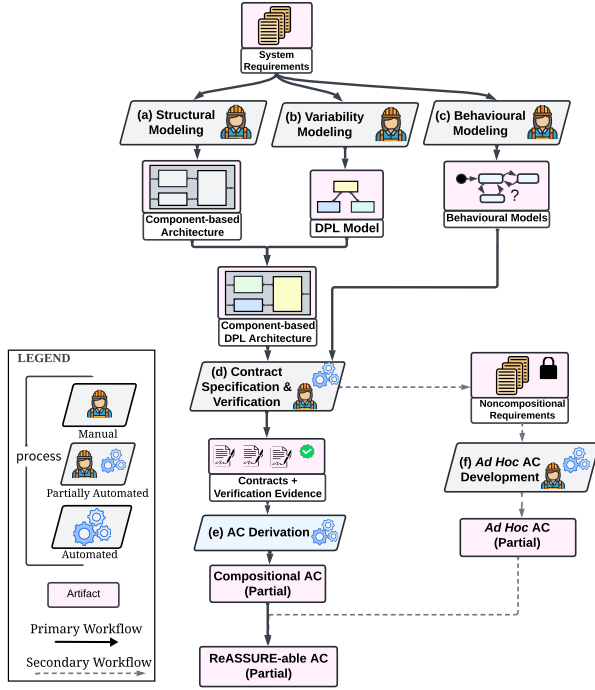


Fig. 4. The design stage of ReASSURE, showing the primary (compositional) and secondary (*ad hoc*) assurance workflows.

based architecture must be defined by providing presence conditions for components and their ports, resulting in a component-based DPL architecture. Based on this architecture, the engineer(s) develop contracts for each of the components which can be used to support as many of the system requirements as possible (Fig. 4d). While the actual specification of contracts must be done manually, analysis of contracts, with respect to properties such as composition and refinement, can be automated. Contract satisfaction for atomic components can be verified using the behavioural models; however, some verification may be inhibited by uncertainty in these models.

Once a set of contracts have been defined and their satisfaction/refinement has been verified, a compositional AC for the managed system can be derived from the architecture model, as outlined in Sec. II-C (Fig. 4e). This derivation is similar to the one defined by Nevsic et al. [16], with one key refinement: the derived AC is *variational*, i.e., is interpreted as a product line of ACs, rather than a single AC for a static product line. Superficially, this means that the presence conditions from the system architecture are also imposed on the corresponding portions of the AC. At a deeper level, this means the AC must be interpreted according to the *variational* AC semantics [6], i.e., goals are to be interpreted as “supported” for some set of products and “unsupported” for all other products.

Example 7. Recall the architecture for the AUV shown in Fig. 3, whose contract-based AC was described in Ex. 5. A portion of this compositional AC, following a period of dynamic assurance, is shown in Fig. 5. Each goal in the

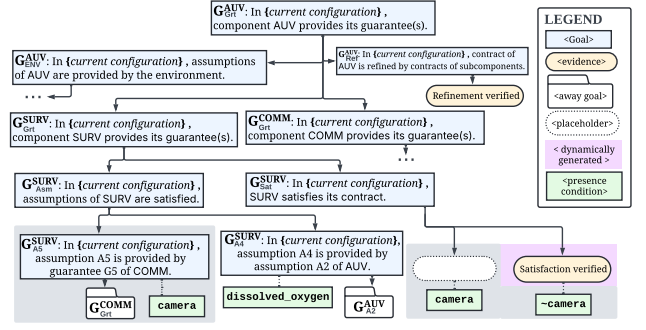


Fig. 5. Fragment of the compositional AC for the AUV, with shaded fragments added alongside the camera feature.

AC is *abstracted* with respect to configurations of the DPL, and presence conditions are used to denote which goals are required for each configuration. For instance, G_{A4}^{SURV} only requires support when the *dissolved_oxygen* feature is active. In this running example, verification of the SURV component requires determining an accurate sensor failure rate, which is postponed until runtime as such, evidence for this goal will be generated dynamically.

We note that some requirements of the SAS may not be feasible to support via contract-based reasoning; we refer to such requirements as *noncompositional*, and their identification is a side-effect of this stage (Fig. 4d). For instance, it may not be possible to soundly refine a system-level requirement into individual contracts for each component. To handle these requirements, ReASSURE splits into a *primary* workflow, which produces a *compositional* AC, and a *secondary* workflow, which produces a “traditional” AC (which we refer to as the *ad hoc* AC).

To support the noncompositional system requirements, the *ad hoc* AC can be developed according to any assurance strategies deemed suitable to the engineers (Fig. 4f). As in ENTRUST, we expect that engineers define AC templates which specify the evidence which will be generated at runtime – however, in ReASSURE, the resulting AC will also be variational. Unlike the compositional AC, in which the completeness of the AC is verified by contract refinement, the completeness of the *ad hoc* template must be determined through other methods, e.g., eliminative argumentation [18]. Also, while the presence conditions for the compositional AC are derived from the system architecture, the presence conditions for the *ad hoc* fragment are defined manually.

The ReASSURE-able AC for the SAS is thus taken as the union of the compositional and *ad hoc* AC fragments. As a final step, in anticipation of potential future assurance regression, each type of evidence in the AC should be assigned a *cost* (e.g., in $\mathbb{N}$), reflecting the relative cost of dynamically producing this evidence. Thereafter, partial verification of the SAS can be undertaken, which will partially instantiate the AC. Once all assurance goals which do not require runtime information are supported, the system can be deployed.

B. Extension: Assurance Evolution & Regression

We now suppose that some period of deployment has

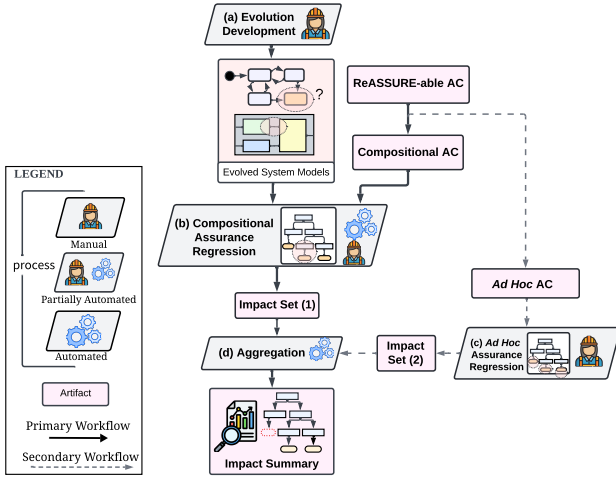


Fig. 6. The evolution stage of ReASSURE.

passed for the SAS, such that some or all of its incomplete assurance goals have been supported. We turn to the *evolution* stage of ReASSURE, the purpose of which is to determine the assurance cost of implementing some evolution of the SAS. The evolution stage of ReASSURE is outlined in Fig. 6. The compositional and *ad hoc* fragments of the ReASSURE-able AC are handled separately in the evolution stage, as the latter requires a more involved regression process. The evolution stage begins with the specification of an evolution by the engineer(s) in terms of the system models (Fig. 6a). We first consider the regression analysis of the compositional AC fragment (Fig. 6b). Since the compositional structure of the AC reflects the system architecture, modifications to a component are kept “local” to that component’s goals, and propagation to other components can be automatically determined through the analysis of contract satisfaction, composition, and refinement (Fig. 6b). For the sake of brevity, here we describe and illustrate only the process for introducing a new feature to an existing component.

If a new feature f is added to component C , this is usually accompanied by some new requirements on the system, and some new port(s) associated with that feature. This will induce another iteration of the contract-based AC development for the new component (finding suitable assumptions, checking satisfaction, etc.) based on its situation within the system architecture. If contract refinement or satisfaction fail, the contracts of components on which C depends may need to be strengthened, inducing additional verification checks for these components. The process iterates until all verification checks have succeeded (possibly requiring re-implementing components). This workflow is highly procedural and is amenable to automation; however, depending on how contract satisfaction is verified for atomic components, manual re-verification by engineers may be required.

Example 8 (Feature Addition). Suppose that, after deploying the AUV, stakeholders decide to add a camera feature. The modified feature model is shown in Fig. 2. Extending the architecture as shown in Fig. 3, we update the contract of

the AUV to also guarantee that the AUV must use the camera to obtain a sufficiently high quality image at least once every 20s. This guarantee is then refined as a guarantee of the extended surveillance component (**G6**). To produce this guarantee, we need to assume that the network bandwidth suffices to send an image every 20s within the communications component, which needs to be guaranteed by the VHF feature (**A5**). The updated version of the compositional AC is shown in Fig. 5. In this case, implementation of the camera feature contains some uncertainty (e.g., the probability of an image being sufficiently high-quality), so we are unable to verify that the component satisfies its contract when the camera is present. However, we can *reuse* the satisfaction of the contract when the camera is absent. Accordingly, we *split* the evidence supporting *surv_contract*; the verification evidence (which previously covered all products) is now annotated by $\sim\text{camera}$, and add a new placeholder for evidence of contract satisfaction once the uncertainty about the camera is resolved. We also include an additional *away* goal *surv_comms_asm_2* expressing the dependency of the camera on the communications component.

Turning now to the *ad hoc* AC fragment, we lose any traceability from system components to assurance goals; thus, in the worst case, every evidence artifact needs to be analyzed for regression (Fig. 6c). In practice, however, only a subset of the configurations of the SAS may be impacted by an evolution; and since the AC goals are annotated by presence conditions, we can automatically skip over any evidence pertaining to configurations which are known not to be affected by the evolution. The regression analysis of individual evidence artifacts can potentially be facilitated using automated analyses, e.g., differential verification [19]; however, in the general compositional process, identifying regression of the *ad hoc* AC will require more involvement from engineers than the compositional AC fragment.

Once the impacted evidence artifacts have been identified, we produce an *impact summary* for the evolution. Intuitively, the impact summary describes the overall cost of reproducing the impacted assurance evidence in terms of the configuration space of the SAS. More formally, the impact summary is defined as a tuple $\langle \phi_{\checkmark}, \phi_{\times}, \phi_{?}, \mathcal{C} \rangle$, where $\phi_{\checkmark}$, $\phi_{\times}$, and $\phi_{?}$ are presence conditions encoding the sets of products for which assurance is preserved, unsatisfiable or incomplete, respectively; the last of these potentially resulting from uncertainty in the evolved SAS model. $\mathcal{C}$ is a variability-aware cost valuation indicating the cost of producing complete assurance for any given product satisfying $\phi_{?}$. The impact summary is produced by automatically propagating the cost of the impacted evidence artifacts “upward” throughout both AC fragments (Fig. 6d). At each branching point in the AC, the costs of assuring each sub-AC are aggregated, beginning from the evidence costs determined at design time.

Example 9 (Impact Summary). After adding the camera feature to the AUV and performing the corresponding assurance evolution, we obtain an impact summary which indicates that no products of the DPL violate their assurance goals,

and that only products with the new camera feature require additional verification at runtime. Symbolically, we have $\phi_{\checkmark} = \sim \text{camera}$, $\phi_{\times} = \text{false}$, $\phi_{?} = \text{camera}$. In this case, all products in the partially assured set have the same evidence artifacts to produce, namely, the evidence of contract satisfaction. In general, the distinct products within $\phi_{?}$ may need to fulfill different types and quantities of evidence.

Thus, the key advantages of ReASSURE are twofold. First, through prioritizing the use of a compositional contract-based AC, structural changes to the SAS can be systematically localized in their assurance impact, which reduces the need for manual regression analysis, and facilitates the reuse of existing assurance artifacts. Second, through using the variational AC formalism, we can directly quantify the costs of re-assuring the system (at design-time and at runtime) in terms of the system's adaptation space. From the impact summary produced in the evolution stage, engineers have the option to restrict certain configurations of the SAS which have been identified as liable to induce significant assurance costs. Alternatively, if engineers are trying to choose one of multiple candidate evolutions, they can use the impact summary to decide which evolution provides the greatest benefit relative to its assurance cost.

IV. TOWARDS IMPLEMENTATION

Our long-term goal is to develop tool support for the modeling and analysis of SASs and their ACs at the design and evolution stages of ReASSURE. Our planned tooling is divided into the *modeling* and the *analysis* layers. The former defines languages and operations necessary for designing DPL-based SASs and ReASSURE-able ACs, and includes four key components: (1) support for variability modeling, i.e., defining product lines of models; (2) compositional modeling, to define modular architectures which will be amenable to compositional assurance; (3) a contract specification language which also supports compositional assurance; (4) an appropriate AC modeling language, such as GSN or CAE. The tool ProFeat [20], which also facilitates family-based analysis, can be used for (1), MMINT-A [6], can be used for (1) and (4). However, to the best of our knowledge, no existing tool or framework combines (1)-(4).

The analysis layer defines all the forms of analysis required for static assurance development and the analysis of evolutions as defined in ReASSURE. We require verification tools, e.g., simulators (e.g., Simulink [21]) and model checkers, e.g., PRISM [22] and ProFeat [20], which can be used to create assurance evidence; support for analyzing relations of contract specifications, such as consistency and refinement (e.g., OCRA [15]); and, if available, the appropriate domain-specific regression analyses used as part of the generic assurance regression defined in Sec. III-B. In implementing ReASSURE, we plan on adapting and integrating these tools, in addition to our own analyses as described in Sec. III-B.

V. RELATED WORK

Assurance of Adaptive Systems. Existing work on conditional safety certification [23], and dynamic assurance [2], [24] are

most closely related to ReASSURE. Both approaches propose to provide runtime assurance for SASs. More recently, safety certification for SASs was picked up by Diemert et al. [25], who consider safety-critical adaptations and propose to extend system-theoretic process analysis [26], an established approach in safety-critical systems engineering, to work with SASs. Finally, work on security assurance [27] aims to design adaptations to provide continuous security guarantees. In contrast to ReASSURE, neither of these approaches handle evolution of assurance as considered here.

Assurance Evolution and Maintenance. Maintaining and repairing assurance cases over system evolutions is a significant problem in safety engineering. ReASSURE follows several works, such as [28], in pursuing a compositional approach to assurance maintenance via contract-based reasoning. Complementary approaches for identifying impacted and reusable elements of ACs include model-based traceability [29], natural language processing [30], and machine learning [31]. Since ReASSURE assumes that only a portion of the SAS's assurance goals are supported compositionally, these techniques can be used to mitigate the complexity of re-assuring the ad hoc fragment of a ReASSURE-able AC. Raeisdanaei et al. considered the maintenance of assurance for automotive systems [32], using variational AC modeling to represent a set of ACs for a fleet of vehicles. Their approach begins by assuming that a set of affected AC nodes has already been identified, and uses syntactic heuristics on the AC to determine the complexity of assurance repair, which could complement the quantitative costs measured during the evolution stage of ReASSURE (Sec. III-B).

Dynamic Product Lines. Several works use DPLs to model runtime variability of SASs, e.g., [10], [11]. Therefore, we argue that the assumption of the SAS being modeled as a DPL is reasonable. However, most of these works do not address the evolution of DPLs. Quinton et al. [33] provide a reference architecture for the evolution of DPLs where they focus on providing a consistent system after an evolution, while they do not consider ACs for evolving DPLs. In [34], the authors propose an approach to model SASs as DPLs and use probabilistic model checking to verify quantitative properties. Since model checking is very costly, especially for DPLs, different static analysis techniques are used in [35] to, e.g., reduce the calls to a model checker. However, these works do not consider ACs of SASs or their evolution.

VI. CONCLUSION

This paper proposed ReASSURE, an extension and refinement of dynamic AC development which aims to systematically maintain assurance of SASs throughout their design evolutions. ReASSURE integrates compositional and variational AC modeling to enable reuse of assurance artifacts and structured regression analysis of SASs whose adaptation is implemented as a DPL with a component-based architecture.

To follow ReASSURE, we require that (i) the managed system of the SAS is modeled (in whole or in part) as a DPL, and (ii) the SAS is modeled by a component-based

architecture supporting contract-based verification. These are stronger assumptions than those made by ENTRUST [2]. However, there has been extensive work using DPLs and component-based design for SASs, so we do not view these requirements as unreasonably restrictive.

In the future, we plan to conduct a full evaluation of ReASSURE over several case studies of SASs from the SEAMS artifact repository, including an AUV [36] and a body sensor network [37], aiming to determine whether the tradeoff between increased design-time cost and reduced complexity at evolution-time is acceptable for a realistic SAS. We also aim to include domain stakeholders tasked with evaluating the costs and benefits of our framework.

REFERENCES

- [1] R. Bloomfield and P. Bishop, “Safety and assurance cases: Past, present and possible future—an Adelard perspective,” in *Making Systems Safer: Proc. of 18th Safety-Critical Systems Symposium*. Springer, 2010, pp. 51–67.
- [2] R. Calinescu, D. Weyns, S. Gerasimou, M. U. Iftikhar, I. Habli, and T. Kelly, “Engineering trustworthy self-adaptive software with dynamic assurance cases,” *IEEE Transactions on Software Engineering*, vol. 44, no. 11, pp. 1039–1069, 2017.
- [3] M. Carwehl, T. Vogel, G. N. Rodrigues, and L. Grunske, “Runtime Verification of Self-Adaptive Systems with Changing Requirements,” in *Proc. of SEAMS’23*, 2023, pp. 104–114.
- [4] T. Chowdhury, E. Lesiuta, K. Rikley, C.-W. Lin, E. Kang, B. Kim, S. Shiraishi, M. Lawford, and A. Wasssyng, “Safe and secure automotive over-the-air updates,” in *Proc. of SAFECOMP’18*. Springer, 2018, pp. 172–187.
- [5] S. Hallsteinsen, M. Hinchey, S. Park, and K. Schmid, “Dynamic Software Product Lines,” in *Systems and Software Variability Management: Concepts, Tools and Experiences*, R. Capilla, J. Bosch, and K.-C. Kang, Eds. Springer, 2013, pp. 253–260.
- [6] L. Murphy, T. Viger, A. D. Sandro, and M. Chechik, “PLACIDUS: Engineering Product Lines of Rigorous Assurance Cases,” in *International Conference on Integrated Formal Methods*. Springer, 2024, pp. 87–108.
- [7] T. Kelly and R. Weaver, “The goal structuring notation—a safety argument notation,” in *Proc. of Dependable Systems and Networks Workshop on Assurance Cases*, vol. 6, 2004.
- [8] S. Varadarajan, R. Bloomfield, J. Rushby, G. Gupta, A. Murugesan, R. Stroud, K. Netkachova, and I. H. Wong, “CLARISSA: Foundations, Tools & Automation for Assurance Cases,” in *Proc. of DASC’23*. IEEE, 2023, pp. 1–10.
- [9] D. Weyns, *An Introduction to Self-Adaptive Systems: A Contemporary Software Engineering Perspective*. John Wiley & Sons, 2020.
- [10] N. Bencomo, P. Sawyer, G. S. Blair, and P. Grace, “Dynamically Adaptive Systems are Product Lines too: Using Model-Driven Techniques to Capture Dynamic Variability of Adaptive Systems,” in *Proc. of SPLC’08*, vol. 2, 2008, pp. 23–32.
- [11] S. Hallsteinsen, E. Stav, A. Solberg, and J. Floch, “Using Product Line Techniques to Build Adaptive Systems,” in *Proc. of SPLC’06*. IEEE, 2006, pp. 141–150.
- [12] D. Batory, “Feature Models, Grammars, and Propositional Formulas,” in *Proc. of SPLC’05*. Springer, 2005, pp. 7–20.
- [13] J. Päßler, M. H. ter Beek, F. Damiani, S. L. T. Tarifa, and E. B. Johnsen, “Formal Modelling and Analysis of a Self-Adaptive Robotic System,” in *Proc. of iFM’23*, ser. LNCS, P. Herber and A. Wijs, Eds., vol. 14300. Springer, 2023, pp. 343–363. [Online]. Available: https://doi.org/10.1007/978-3-031-47705-8_18
- [14] A. Cimatti and S. Tonetta, “Contracts-refinement proof system for component-based embedded systems,” *Science of computer programming*, vol. 97, pp. 333–348, 2015.
- [15] A. Cimatti, M. Dorigatti, and S. Tonetta, “OCRA: A tool for checking the refinement of temporal contracts,” in *Proc. of ASE’13*. IEEE, 2013, pp. 702–705.
- [16] D. Nešić, M. Nyberg, and B. Gallina, “Product-line assurance cases from contract-based design,” *Journal of Systems and Software*, vol. 176, p. 110922, 2021.
- [17] E. B. Gil, R. Caldas, A. Rodrigues, G. L. G. da Silva, G. N. Rodrigues, and P. Pelliccione, “Body Sensor Network: A Self-Adaptive System Exemplar in the Healthcare Domain,” in *Proc. of SEAMS’21*. IEEE, 2021, pp. 224–230.
- [18] J. B. Goodenough, C. B. Weinstock, and A. Z. Klein, “Eliminative argumentation: A basis for arguing confidence in system properties,” *Software Engineering Institute, Carnegie Mellon University, Pittsburgh, PA, Tech. Rep. CMU/SEI-2015-TR-005*, 2015.
- [19] A. Wang, N. Feng, and M. Chechik, “Code-Level Functional Equivalence Checking of Annotative Software Product Lines,” in *Proc. of SPLC’23, Volume A*, 2023, pp. 64–75.
- [20] P. Chrszon, C. Dubslaff, S. Klüppelholz, and C. Baier, “ProFeat: Feature-Oriented Engineering for Family-Based Probabilistic Model Checking,” *Formal Aspects Comput.*, vol. 30, no. 1, pp. 45–75, 2018. [Online]. Available: <https://doi.org/10.1007/s00165-017-0432-4>
- [21] C. Heinzemann, J. Rieke, and W. Schäfer, “Simulating self-adaptive component-based systems using MATLAB/Simulink,” in *Proc. of SEAMS’13*. IEEE, 2013, pp. 71–80.
- [22] M. Kwiatkowska, G. Norman, and D. Parker, “PRISM: Probabilistic symbolic model checker,” in *International Conference on Modelling Techniques and Tools for Computer Performance Evaluation*. Springer, 2002, pp. 200–204.
- [23] D. Schneider and M. Trapp, “Conditional safety certification of open adaptive systems,” *ACM Transactions on Autonomous and Adaptive Systems (TAAS)*, vol. 8, no. 2, pp. 1–20, 2013.
- [24] E. Asaadi, E. Denney, J. Menzies, G. J. Pai, and D. Petroff, “Dynamic assurance cases: a pathway to trusted autonomy,” *Computer*, vol. 53, no. 12, pp. 35–46, 2020.
- [25] S. Diemert and J. Weber, “Safety-Critical Adaptation in Self-Adaptive Systems,” in *Proc. of ISSREW’22*. IEEE, 2022, pp. 371–380.
- [26] S. Diemert and J. H. Weber, “Hazard Analysis for Self-Adaptive Systems Using System-Theoretic Process Analysis,” in *Proc. of SEAMS’23*, 2023, pp. 145–156.
- [27] M. Riegler, J. Sametinger, and M. Vierhauser, “A Distributed MAPE-K Framework for Self-Protective IoT Devices,” in *Proc. of SEAMS’23*, 2023, pp. 202–208.
- [28] F. Warg, H. Blom, J. Borg, and R. Johansson, “Continuous deployment for dependable systems with continuous assurance cases,” in *Proc. of ISSREW’19*. IEEE, 2019, pp. 318–325.
- [29] R. Shahin, S. Kokaly, and M. Chechik, “Towards Certified Analysis of Software Product Line Safety Cases,” in *Proc. of SAFECOMP’21*. Springer, 2021, pp. 130–145.
- [30] F. U. Muram and M. A. Javed, “Attest: Automating the review and update of assurance case arguments,” *Journal of systems architecture*, vol. 134, p. 102781, 2023.
- [31] C. Menghi, T. Viger, A. Di Sandro, C. Rees, J. Joyce, and M. Chechik, “Assurance case development as data: A manifesto,” in *Proc. of ICSE-NIER’23*. IEEE, 2023, pp. 135–139.
- [32] A. Raeisdanaei, L. Murphy, A. Di Sandro, M. Askarpour, T. Viger, and M. Chechik, “Evaluation of Automotive OTA Updates Using Assurance Cases,” in *Proc. of MODELS’24 Workshops*, 2024, pp. 720–724.
- [33] C. Quinton, M. Vierhauser, R. Rabiser, L. Baresi, P. Grünbacher, and C. Schuhmayer, “Evolution in dynamic software product lines,” *Journal of Software: Evolution and Process*, vol. 33, no. 2, pp. e2293:1–e2293:25, 2021.
- [34] C. Dubslaff, C. Baier, and S. Klüppelholz, “Probabilistic Model Checking for Feature-Oriented Systems,” in *Transactions on Aspect-Oriented Software Development XII*, ser. LNCS, S. Chiba, É. Tanter, E. Ernst, and R. Hirschfeld, Eds. Germany: Springer, 2015, vol. 8989, pp. 180–220.
- [35] H. Göttmann, I. Bacher, N. Gottwald, and M. Lochau, “Static Analysis Techniques for Efficient Consistency Checking of Real-Time-Aware DSPL Specifications,” in *Proc. of VaMoS’21*, 2021.
- [36] G. R. Silva, J. Päßler, J. Zwanepol, E. Alberts, S. L. T. Tarifa, I. Gerostathopoulos, E. B. Johnsen, and C. H. Corbato, “SUAVE: An Exemplar for Self-Adaptive Underwater Vehicles,” in *Proc. of SEAMS’23*. IEEE, 2023, pp. 181–187.
- [37] M. Carwehl and W. Reisig, “Modeling classes of body sensor networks,” in *Modellierung 2024*. Gesellschaft für Informatik eV, 2024, pp. 65–82.